

Cache-Oriented Performance Opportunities in Pioneer.jl

Prepared for Pioneer.jl development

May 18, 2026

Context

This note maps the main lessons from Algorithmica’s “RAM & CPU Caches” chapter to the current Pioneer.jl codebase. The relevant cache concepts are: bandwidth vs. latency, cache-line traffic, memory-level parallelism, hardware prefetching, structure layout, cache associativity, TLB/page behavior, and AoS-vs-SoA layout.

Pioneer already uses several of these ideas in its performance-critical code: compact fragment records, struct-of-arrays fragment-bin metadata, partition-local `UInt16` precursor IDs, partition-major traversal, fused match/build loops, and packed metadata in the fused sparse matrix. The best opportunities now are not broad rewrites. They are targeted reductions in avoidable memory traffic, temporary allocation, and pointer chasing.

Priority Summary

- **Precompute top-N intensity thresholds once per scan.** Low risk. Removes repeated scans and partial sorts when one scan overlaps multiple fragment-index partitions.
- **Avoid per-scan candidate allocation in fused search.** Low risk. Removes hot-loop allocation and copy work while preserving the existing algorithm.
- **Repack PSM accumulator structs.** Medium risk. Hot records are currently about 72 bytes, just over one 64-byte cache line. Better packing can reduce cache-line pressure.
- **Benchmark dense or generation-stamped precursor maps.** Medium risk. Current Dict-backed maps avoid large per-thread arrays but introduce pointer chasing in scan scoring.
- **Specialize simple mass-error peak matching.** Medium risk. The general path loads three peak-window arrays; simple models may need fewer streams.
- **Use fixed-size isotope scratch storage.** Medium risk. Tiny vector scratch is repeatedly filled and read inside the fragment inner loop.

1 Precompute Top-N Intensity Thresholds Once Per Scan

Where

The partitioned fragment-index search computes an optional top-N peak intensity threshold inside `_run_thread`:

- `src/structs/SpectralLibrary/PartitionedFragmentIndex/search.jl:519`
- threshold extraction around `search.jl:552`
- `partialsort!` call around `search.jl:559`

This sits inside the partition-major loop. A single scan can be relevant to more than one precursor-m/z partition. When that happens, its intensity array is visited and partially sorted once per relevant partition.

Cache concept

This is a cache-line bandwidth issue. The intensity array is contiguous and therefore friendly to hardware prefetching, but repeatedly rereading it still consumes memory bandwidth and displaces other hot data. The `partialsort!` also mutates a per-thread scratch buffer and can touch a substantial fraction of the scan's peak intensities.

Implementation sketch

Move the threshold calculation into the existing per-scan precompute phase in `searchFragmentIndexPartitionMajorH`. The function already computes:

- scan iRT lower/upper bounds,
- scan precursor m/z bounds,
- scan center iRT.

Add a `Vector{Float32}` named something like `scan_intensity_thresholds`. If `max_peaks==0`, fill it with `0.0f0`. Otherwise, compute the threshold once per `all_scan_idx`s entry before the partition-major parallel loop. Then pass this vector into `_run_thread` and read:

```
intensity_threshold = scan_intensity_thresholds[si]
```

inside the scan loop.

Expected benefit

The win should scale with:

- the number of scans that overlap multiple partitions,
- the number of peaks per scan,
- the number of active threads,
- whether `max_peaks>0`.

This should reduce both runtime and transient memory traffic during wide scout or other top-N filtered index searches. The behavior should be exactly equivalent if the threshold definition is preserved.

Risks

The main risk is memory overhead for storing one threshold per scan, which is small. A secondary risk is moving work from per-thread scratch into a shared precompute stage. That should be fine because each scan threshold is independent.

Benchmark

Use a representative file and compare:

- total time in `searchFragmentIndexPartitionMajorHinted`,
- allocations,
- number of threshold computations, if instrumented,
- candidate count equality.

The candidate output should be bit-identical because only redundant threshold calculation is removed.

2 Avoid Per-Scan Candidate Allocation in Fused Search

Where

In the fused scan path:

- `src/Routines/SearchDIA/process_scans_fused.jl:21`
- `collect(Int64,prec_range)` around `process_scans_fused.jl:146`
- optional target/decoy split buffers around `process_scans_fused.jl:134`

The normal non-split path materializes a fresh `Vector{Int64}` from `prec_range` before calling `run_fused!`. But `run_fused!` already accepts an `AbstractVector{Int64}` for `prec_range`, and the loop body does not require ownership of the candidate index list.

Cache concept

This combines bandwidth and allocation overhead. Allocating and filling a vector per scan turns an already available range into fresh memory traffic. It also adds pressure on Julia's allocator and garbage collector.

Implementation sketch

There are two practical routes:

1. Broaden `run_fused!`'s `prec_range` type to accept `AbstractUnitRange{<:Integer}` or simply remove the concrete `AbstractVector{Int64}` restriction.
2. Call `_run_subset!(prec_range)` directly in the non-split branch.

For the split branch, keep two reusable buffers on the per-thread search context or fused scratch object, for example `target_idx_scratch` and `decoy_idx_scratch`. Use `empty!` between scans. That keeps the split feature allocation-free after warmup.

Expected benefit

This is likely a small but reliable win. It should show up most clearly in files with many MS2 scans and modest candidate counts where allocation overhead is not drowned out by deconvolution time.

Risks

Low. The non-split path should preserve behavior exactly. For split mode, the main risk is accidentally retaining stale entries if buffers are not cleared. That is easy to test.

Benchmark

Run the current unit tests around fused scanning, then compare allocation counts for a representative search with:

```
@time  
@allocated
```

or the existing profiling hooks. Confirm identical PSM output for a fixed input.

3 Repack PSM Accumulator and Output Structs

Where

The hot PSM records are:

- `MainUnscoredPSM{Float32}` in `src/Routines/SearchDIA/PSMs/UnscoredPSMs.jl:27`
- `TuningUnscoredPSM{Float32}` in the same file
- `MainSearchScoredPSM{Float32,Float16}` in `src/Routines/SearchDIA/PSMs/ScoredPSMs.jl:70`
- `TuningScoredPSM{Float32,Float16}` in the same file

A quick size check showed:

- `MainUnscoredPSM{Float32}`: 72 bytes,
- `TuningUnscoredPSM{Float32}`: 36 bytes,
- `MainSearchScoredPSM{Float32,Float16}`: 72 bytes,
- `TuningScoredPSM{Float32,Float16}`: 40 bytes.

The two main-search structs cross the 64-byte cache-line boundary.

Cache concept

CPU caches move data in cache lines, commonly 64 bytes. A 72-byte record often requires two cache lines when accessed in an array, even if the code only needs part of the record. That increases bandwidth use and lowers effective cache capacity.

Implementation sketch

First, reorder fields to group by alignment:

- all `Float32` fields together,
- all `Float16` fields together,
- all `UInt32` fields together,
- all `UInt8` fields together.

Then consider packing low-cardinality counters:

- several `UInt8` counts can be packed into one `UInt32`,
- boolean/rank flags can be packed into bit fields represented by `UInt16` or `UInt32`,
- top-rank intensities should stay as separate `Float32` fields if downstream code reads them directly.

This should be done with constructors and accessor functions so the scoring code does not become unreadable. The current `CompactFrag` design is a good local pattern: it packs metadata but keeps inline getters.

Expected benefit

If `MainUnscoredPSM` can be brought to 64 bytes or less, updates in `apply_main_scoring!` and reads in `Score!` should touch fewer cache lines. If `MainSearchScoredPSM` can also be reduced, output construction and later ML feature-table conversion may benefit.

Risks

This is riskier than the first two ideas because these structs are part of internal data contracts. Field order changes can affect Arrow output if any code relies on Tables.jl's struct field order. Packing also adds getter/setter logic, which can make code harder to maintain.

The safe path is:

1. add size tests using `sizeof`,
2. preserve public column names in output conversion,
3. benchmark before and after,
4. avoid packing fields that are read constantly unless the getter inlines cleanly.

Benchmark

Benchmark:

- fused scan runtime,
- `Score!` runtime,
- PSM output equality,
- `sizeof` for each struct,
- allocations and compilation impact.

4 Benchmark Dense or Generation-Stamped Precursor Maps

Where

The scan-local precursor map abstraction is in:

- `src/structs/PrecursorMap.jl`
- dense backend around `PrecursorMap.jl:57`
- sparse Dict-backed backend around `PrecursorMap.jl:159`
- current search context field `id_to_col::SparsePrecMap{UInt16}` in `src/Routines/SearchDIA/SearchMethods/SearchTypes.jl:196`

Cache concept

Julia `Dict` lookup is flexible and memory-efficient for sparse active sets, but it involves hashing and indirect memory access. That can introduce pointer chasing, which is exactly the kind of access pattern that defeats memory prefetching and exposes memory latency.

A dense vector indexed by precursor ID has excellent locality for each lookup, but it can be large per thread. The repo comments already note that a fully dense map can cost substantial memory on large libraries.

Implementation sketch

Benchmark three variants behind the existing `AbstractPrecursorMap` interface:

1. current `SparsePrecMap`;
2. existing `DensePrecMap`;
3. generation-stamped dense map.

A generation-stamped dense map would store:

- `vals::Vector{V}`,
- `stamp::Vector{UInt16}` or `Vector{UInt32}`,
- current generation counter,
- active key list.

Reset becomes:

```
generation += 1
n_active = 0
```

Instead of zeroing active entries. A key is present if its stamp equals the current generation. This is useful when the dense vector is acceptable but reset or active tracking should be cheap.

Expected benefit

This could speed up:

- `id_to_col[precursor_idx]` in `Score!`,
- `id_to_col[match_column_id(...)]` in `run_fused!`,
- warm-start weight transfer in deconvolution utilities.

The expected benefit is workload-dependent. For very large libraries and many threads, the dense memory footprint may hurt more than it helps. For moderate libraries or machines with enough memory, it may remove a latency-heavy hash lookup from hot paths.

Risks

The main risk is per-thread memory growth and lower effective cache capacity. This is exactly the tradeoff highlighted by the cache chapter: a theoretically faster direct lookup can become slower if it forces a much larger working set.

Benchmark

Make backend selection configurable by an environment variable or parameter, then compare:

- fused search runtime,
- max resident memory,
- GC time,
- output equality.

The result should be interpreted by library size and thread count, not as a single universal winner.

5 Specialize Simple Mass-Error Peak Matching

Where

The fused path precomputes peak matching windows in:

- `prepare_scan_peaks!` at `src/Routines/SearchDIA/CommonSearchUtils/fusedMatch.jl:417`
- SIMD matching in `scan_for_nearest_in_window` at `fusedMatch.jl:476`

The current general path stores and scans three contiguous arrays:

- corrected peak m/z ,
- lower observable bound,
- upper observable bound.

This is a good SoA design for the general intensity-aware mass-error model.

Cache concept

SoA helps SIMD because each vector load reads one field across adjacent peaks. But every extra array is another memory stream. The current matching loop can load three 32-byte vectors for every eight peaks. If a simpler mass-error model does not need precomputed per-peak bounds, one or two of those streams may be avoidable.

Implementation sketch

Keep the general path unchanged. Add a specialized method for simple mass-error models where the tolerance can be computed directly from the theoretical m/z or corrected m/z without reading `obs_low` and `obs_high`.

For example:

- for a fixed ppm model, search by corrected m/z and compare absolute error to `iso_mz*ppm*1e-6`;
- for a constant Da model, compare absolute error to the constant bound;
- keep the intensity-aware model on the existing three-array path.

This would be multiple dispatch on the concrete mass-error model type, not a runtime branch inside the hottest loop.

Expected benefit

For simple models, this can reduce peak-window scanning bandwidth by up to two array streams. It may also reduce the per-scan precompute work in `prepare_scan_peaks!`. The effect should be largest on scans with many peaks and many candidate fragment isotope windows.

Risks

The semantic risk is high if the specialized path does not exactly match the mass-error model's intended acceptance window. The current code explicitly preserves intensity-dependent bias and tolerance. That behavior must stay untouched for those models.

The implementation should therefore start with a very narrow specialization for one simple model and an equivalence test against the current path.

Benchmark

For a fixed simple model:

- compare matched peak indices and ppm errors against the current path,
- measure fused matching time,
- inspect allocation and cache behavior if hardware counters are available.

6 Use Fixed-Size Isotope Scratch Storage

Where

Fragment isotope intensities are computed in:

- `src/Routines/SearchDIA/CommonSearchUtils/getFragIsotopes.jl:17`
- called inside `run_fused!` around `src/Routines/SearchDIA/CommonSearchUtils/fusedMatch.jl:818`

The search context currently stores `isotopes::Vector{Float32}` and `precursor_transmission::Vector{Float32}`.

Cache concept

The isotope buffers are tiny, but they sit in the innermost fragment loop. The current code repeatedly fills and reads a heap-allocated vector. That is not a large memory object, but the operation is frequent. Fixed-size storage can keep more of the state in registers or stack-like local storage, improving locality and reducing bounds checks when the compiler can see the size.

Implementation sketch

There are two levels of aggressiveness:

1. Replace `fill!(frag_isotopes,0)` with loops only over the active isotope range where safe. This is the conservative option.
2. Introduce a fixed-size representation for the common maximum isotope count, for example an `NTuple` or `MVector` style buffer if `StaticArrays` is already acceptable in the hot path.

The second route needs care. `StaticArrays` can help for tiny fixed-size data, but using them incorrectly can increase register pressure or compilation time.

Expected benefit

This is likely a small inner-loop win rather than a large algorithmic gain. It may matter because `getFragIsotopes!` is called for every candidate fragment that passes rank filters.

Risks

The main risk is increasing code complexity in a numerically sensitive area. Another risk is creating too many method specializations if isotope count becomes a type parameter across search kinds.

Benchmark

Microbenchmark `getFragIsotopes!` for common fragment and precursor cases, then benchmark full fused search. Require bitwise or near-bitwise equality for isotope intensities and PSM results.

General Benchmarking Plan

For each candidate, use a two-layer benchmark:

1. A focused microbenchmark for the changed function.
2. An end-to-end representative search file.

Track:

- wall time,
- allocations,
- output equality or accepted numerical tolerance,
- resident memory,
- number of candidates, matches, misses, and PSMs.

If available, hardware counters would be ideal for the cache-specific claims: L1/L2/L3 misses, LLC load misses, branch misses, and cycles. Without counters, allocation counts and stable end-to-end timing are still enough to decide whether a change is worth keeping.

Suggested Order of Work

1. Implement top-N threshold precompute. It is low risk and likely measurable.
2. Remove fused per-scan candidate allocation. This is straightforward and should reduce allocation noise.
3. Add a backend switch for precursor maps and benchmark sparse vs. dense vs. generation-stamped dense.
4. Repack PSM structs only after the first two changes are measured.
5. Try simple-model peak matching specialization if profiling shows peak matching remains dominant.
6. Try fixed-size isotope scratch last, because it is the most inner-loop specific and easiest to overfit.

References

- Algorithmica, RAM & CPU Caches: <https://en.algorithmica.org/hpc/cpu-cache/>
- Memory bandwidth: <https://en.algorithmica.org/hpc/cpu-cache/bandwidth/>
- Memory latency: <https://en.algorithmica.org/hpc/cpu-cache/latency/>
- Cache lines: <https://en.algorithmica.org/hpc/cpu-cache/cache-lines/>

- Memory-level parallelism and prefetching: <https://en.algorithmica.org/hpc/cpu-cache/mlp/>, <https://en.algorithmica.org/hpc/cpu-cache/prefetching/>
- Alignment, associativity, paging, and AoS/SoA: <https://en.algorithmica.org/hpc/cpu-cache/alignment/>, <https://en.algorithmica.org/hpc/cpu-cache/associativity/>, <https://en.algorithmica.org/hpc/cpu-cache/paging/>, <https://en.algorithmica.org/hpc/cpu-cache/aos-soa/>